



Proyecto Final Data Science

“Modelo de clasificación para predecir
lluvias en forma temprana”

Primera entrega

Alumno: Foletto Mariano

Comisión: 90455

Profesor: Jorge Ruiz

Tutor: Luciano Lisachi

Índice

Problemática por resolver	3
Solución propuesta.....	3
Modelado	3
Data Acquisition	3
Descripción de variables y dominio.....	4
Estadística descriptiva	4
Data Wrangling y EDA	6
Tratamiento de valores nulos	6
Variables numéricas	7
Correlación sin encodear (variables numéricas)	10
Feature Engineering.....	10
Correlación con variables encodeadas.....	12
Baseline	12
Test - Train - Validation Split	12
Métricas	13
Conclusiones primera iteración	14
Segunda iteración.....	14
Modelos por implementar	15
Resultados obtenidos y comparativa de métricas.....	15
Función de pérdida sobre Train y Val.....	15
Accuracy sobre Train, Val y Test	15
Recall sobre Val y Test (clase 1)	16
ROC/AUC sobre Test incluyendo umbrales	16
Análisis PCA	21
Corrección del umbral (optimal cutoff)	22
Mejoras para iteraciones posteriores	23
Conclusiones segunda iteración	23
Tercera iteración	25
Modelos optimizados y métricas.....	25
GridSearchCV utilizando StratifiedKfold	25
RandomizedSearchCV utilizando StratifiedKfold	27
Modelos boosting	28
Modelos boosting optimizados.....	31
.....	31

Bagging (Bootstrap aggregating)	32
Bagging optimizado.....	34
Conclusiones tercera iteración	35

Problemática por resolver

Analizar datos históricos de lluvias, periodo 1/11/2007 hasta 25/6/2017, y lograr predecir en forma temprana si hay lluvia al día siguiente. Se puede suponer que se trabaja para un privado y/o estatal, que necesite prever las lluvias para la toma de decisiones.

Solución propuesta

Se propone realizar un modelo de clasificación binaria y/o regresión, realizando previamente un análisis de los datos persistidos en la base, tanto de la integridad de estos como de su poder predictivo para la variable objetivo a inferir. Se seleccionará aquel modelo que no solo brinde el mayor accuracy sobre el set de validación, sino que se prestará especial atención en el valor del recall buscando que la misma tienda a 1 (uno) o -lo que es lo mismo- que los falsos negativos tiendan a cero.

$$Recall = TP / (TP + FN)$$

$$Accuracy = TP + TN / (TP + TN + FN + FP)$$

Modelado

Data Acquisition

Una vez obtenido el acceso a la base, se volcó el contenido de esta a un archivo .csv el cuál se encuentra en github ([Link](#)). El usuario de los datos en cuestión nos brindó una descripción de las variables presentes en el mismo, como así también el dominio o rango esperado de sus variables numéricas, en caso de que haya algún tipo de error en la base de datos. En una primera etapa se le prestará especial atención a aquellas variables que contengan valores considerados absurdos e imposibles para un ser humano.

Descripción de variables y dominio

- **Date:** Fecha de la observación [1/11/2007 hasta 25/6/2017].
- **Location:** Nombre común de la ubicación de la estación meteorológica [49 location].
- **MinTemp:** Temperatura mínima en grados Celsius [rango de -8,5 a 33,9].
- **MaxTemp:** Temperatura máxima en grados Celsius [rango de -4,8 a 48,1].
- **Rainfall:** Cantidad de lluvia registrada en el día (mm) [rango de 0 a 371].
- **Evaporation:** Evaporación del tipo Clase A (mm) en las 24 horas hasta las 9am [rango de 0 a 145].
- **Sunshine:** Número de horas de sol brillante en el día [rango de 0 a 14,5].
- **WindGustDir:** Dirección de la ráfaga de viento más fuerte en las 24 horas hasta la medianoche [E, ENE, ESE, N, NE, NNE, NNW, NW, S, SE, SSE, SSW, SW, W, WNW, WSW].
- **WindGustSpeed:** Velocidad (km/h) de la ráfaga de viento más fuerte en las 24 horas hasta la medianoche [rango de 6 a 135].
- **WindDir9am:** Dirección del viento a las 9am [E, ENE, ESE, N, NE, NNE, NNW, NW, S, SE, SSE, SSW, SW, W, WNW, WSW].
- **WindDir3pm:** Dirección del viento a las 3pm [E, ENE, ESE, N, NE, NNE, NNW, NW, S, SE, SSE, SSW, SW, W, WNW, WSW].
- **WindSpeed9am:** Velocidad del viento (km/h) promedio en los 10 minutos previos a las 9am [rango de 0 a 130].
- **WindSpeed3pm:** Velocidad del viento (km/h) promedio en los 10 minutos previos a las 3pm [rango de 0 a 87].
- **Humidity9am:** Humedad (porcentaje) a las 9am [rango de 0 a 100].
- **Humidity3pm:** Humedad (porcentaje) a las 3pm [rango de 0 a 100].
- **Pressure9am:** Presión atmosférica (hPa) reducida al nivel del mar a las 9am [rango de 980,5 a 1041].
- **Pressure3pm:** Presión atmosférica (hPa) reducida al nivel del mar a las 3pm [rango de 977,1 a 1039,6].
- **Cloud9am:** Fracción del cielo cubierta por nubes a las 9am (en oktas: octavos) [rango de 0 a 9].
- **Cloud3pm:** Fracción del cielo cubierta por nubes a las 3pm (en oktas: octavos) [rango de 0 a 9].
- **Temp9am:** Temperatura (°C) a las 9am [rango de -7,2 a 40,2].
- **Temp3pm:** Temperatura (°C) a las 3pm [rango de -5,4 a 46,7].
- **RainToday:** Booleano: 1 si la precipitación (mm) en las 24 horas hasta las 9am supera 1mm, de lo contrario 0 [posee "Yes" y "No" según corresponda precipitación mayor a 1mm].
- **RainTomorrow:** Cantidad de lluvia del día siguiente (mm), usada para crear la variable de respuesta RainTomorrow [posee "Yes" y "No" según corresponda precipitación mayor a 1mm].

Estadística descriptiva

- El dataset obtenido de la base de datos posee 145.560 filas y 23 columnas, de las cuales 22 (veintidós) son variables independientes para analizar y 1 (una) corresponde a la variable target o dependiente.
- El dataset no ni filas repetidas.

- El dataset posee valores nulos, en el dataset figuran como "NA". Estos se tratarán posteriormente.
- De las 23 (veintitrés) variables independientes, 6 (seis) son categóricas (Date, Location, WindGustDir, WindGustDir, WindDir9am, WindDir3pm) y 16 (dieciséis) son numéricas (MinTemp, MaxTemp, Rainfall, Evaporation, Sunshine, WindGustSpeed, WindSpeed9am, WindSpeed3pm, Humidity9am, Humidity3pm, Pressure9am, Pressure3pm, Cloud9am, Cloud3pm, Temp9am, Temp3pm). La variable RainTomorrow (target) y RainToday ya se encuentran binarizada.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  145460 non-null object
1   Location              145460 non-null object
2   MinTemp               143975 non-null float64
3   MaxTemp               144199 non-null float64
4   Rainfall              142199 non-null float64
5   Evaporation           82670 non-null float64
6   Sunshine              75625 non-null float64
7   WindGustDir           135134 non-null object
8   WindGustSpeed         135197 non-null float64
9   WindDir9am            134894 non-null object
10  WindDir3pm            141232 non-null object
11  WindSpeed9am          143693 non-null float64
12  WindSpeed3pm          142398 non-null float64
13  Humidity9am           142806 non-null float64
14  Humidity3pm           140953 non-null float64
15  Pressure9am           130395 non-null float64
16  Pressure3pm           130432 non-null float64
17  Cloud9am              89572 non-null float64
18  Cloud3pm              86102 non-null float64
19  Temp9am               143693 non-null float64
20  Temp3pm               141851 non-null float64
21  RainToday             142199 non-null object
22  RainTomorrow          142193 non-null object
dtypes: float64(16), object(7)
memory usage: 25.5+ MB
```

Figura 1: Información del dataset.

	count	unique	top	freq
Date	145460	3436	2017-06-24	49
Location	145460	49	Canberra	3436
WindGustDir	135134	16	W	9915
WindDir9am	134894	16	N	11758
WindDir3pm	141232	16	SE	10838
RainToday	142199	2	No	110319
RainTomorrow	142193	2	No	110316

Figura 2: Estadística descriptiva básica de las variables categóricas.

	count	mean	std	min	25%	50%	75%	max
MinTemp	143975.0	12.194034	6.398495	-8.5	7.6	12.0	16.9	33.9
MaxTemp	144199.0	23.221348	7.119049	-4.8	17.9	22.6	28.2	48.1
Rainfall	142199.0	2.360918	8.478060	0.0	0.0	0.0	0.8	371.0
Evaporation	82670.0	5.468232	4.193704	0.0	2.6	4.8	7.4	145.0
Sunshine	75625.0	7.611178	3.785483	0.0	4.8	8.4	10.6	14.5
WindGustSpeed	135197.0	40.035230	13.607062	6.0	31.0	39.0	48.0	135.0
WindSpeed9am	143693.0	14.043426	8.915375	0.0	7.0	13.0	19.0	130.0
WindSpeed3pm	142398.0	18.662657	8.809800	0.0	13.0	19.0	24.0	87.0
Humidity9am	142806.0	68.880831	19.029164	0.0	57.0	70.0	83.0	100.0
Humidity3pm	140953.0	51.539116	20.795902	0.0	37.0	52.0	66.0	100.0
Pressure9am	130395.0	1017.649940	7.106530	980.5	1012.9	1017.6	1022.4	1041.0
Pressure3pm	130432.0	1015.255889	7.037414	977.1	1010.4	1015.2	1020.0	1039.6
Cloud9am	89572.0	4.447461	2.887159	0.0	1.0	5.0	7.0	9.0
Cloud3pm	86102.0	4.509930	2.720357	0.0	2.0	5.0	7.0	9.0
Temp9am	143693.0	16.990631	6.488753	-7.2	12.3	16.7	21.6	40.2
Temp3pm	141851.0	21.683390	6.936650	-5.4	16.6	21.1	26.4	46.7

Figura 3: Estadística descriptiva básica de las variables numéricas.

Data Wrangling y EDA

Tratamiento de valores nulos

En esta primera iteración no las descarto y le imputo valores en el código posterior (otra posibilidad a evaluar es descartando estas variables que tienen muchos faltantes: Sunshine, Evaporation, Cloud3pm y Cloud9am).

```

Sunshine      48.01
Evaporation   43.17
Cloud3pm      40.81
Cloud9am      38.42
Pressure9am    10.36
Pressure3pm    10.33
WindDir9am     7.26
WindGustDir    7.10
WindGustSpeed  7.06
Humidity3pm    3.10
WindDir3pm     2.91
Temp3pm        2.48
RainTomorrow   2.25
Rainfall       2.24
RainToday      2.24
WindSpeed3pm   2.11
Humidity9am    1.82
WindSpeed9am   1.21
Temp9am        1.21
MinTemp        1.02
MaxTemp        0.87
Date           0.00
Location       0.00
dtype: float64

```

Figura 4: Análisis del porcentaje de faltante en cada columna

Transformación de la información: primero se pasó la columna "date" a formato fecha. Se agrego la columna mes para agrupar por posible estacionalidad y usarla para la imputación de valores numéricos.

Estrategia de imputación:

- ✓ Numéricas por mediana de grupo
- ✓ Categóricas por moda de grupo (excepto direcciones)
- ✓ Relleno residual (si aún quedó algo, por ejemplo, direcciones del viento)

Variables numéricas

Se dispuso a graficar la distribución y el boxplot de cada una, habiendo filtrado previamente los valores absurdos o irreales encontrados mediante el análisis descriptivo. Se adjuntan gráficos de aquellas que posean outliers su posterior interpretación.

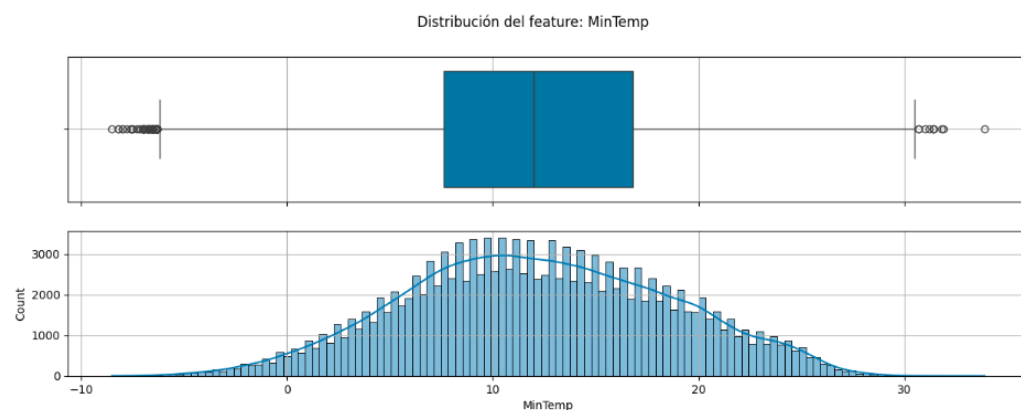


Figura 5

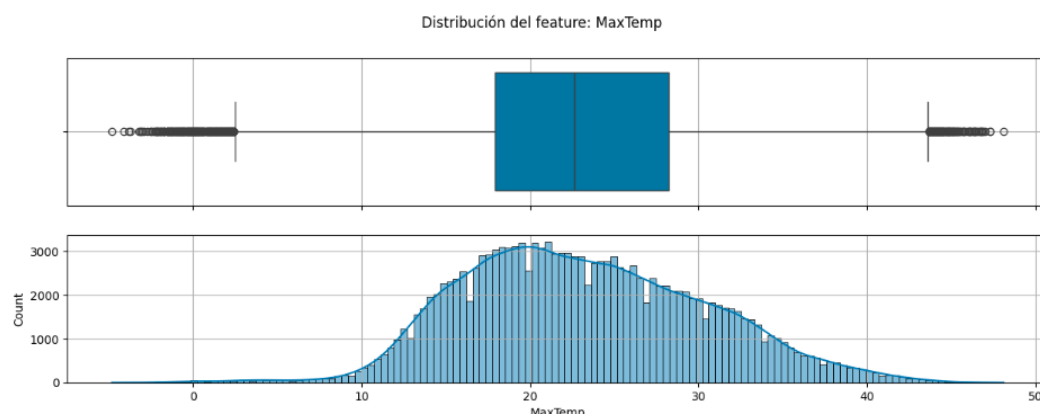


Figura 6

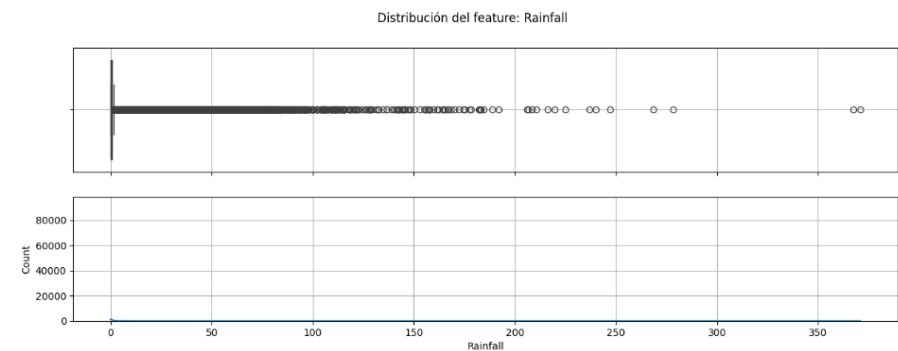


Figura 7

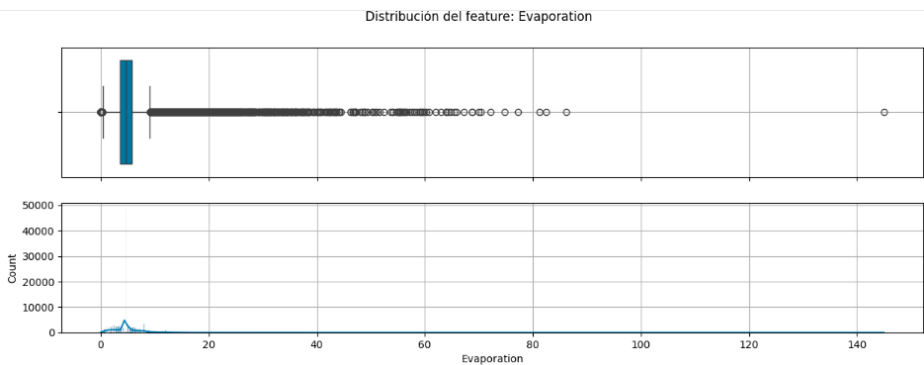


Figura 8

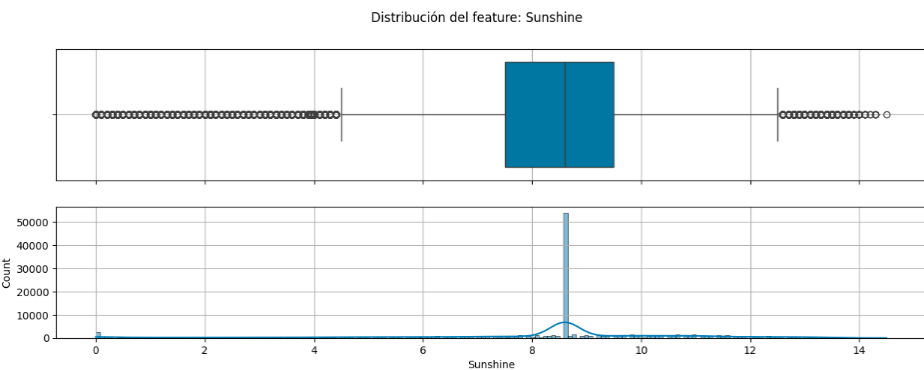


Figura 9

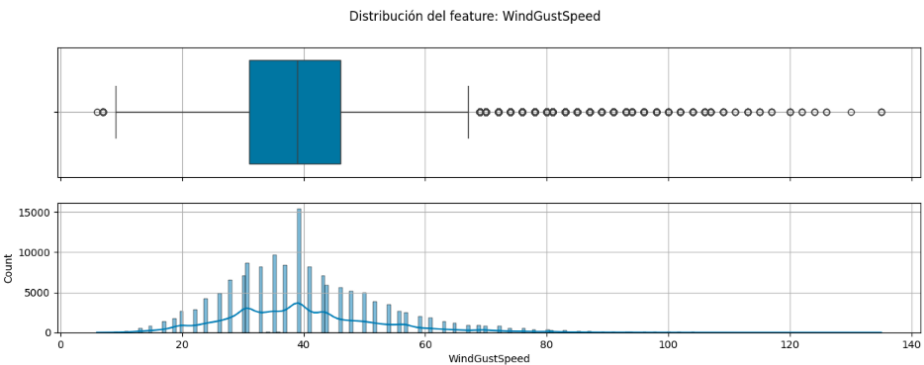


Figura 10

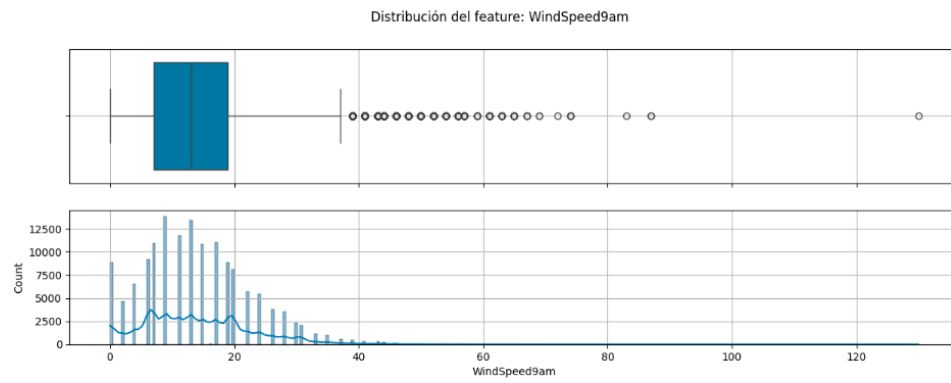


Figura 11

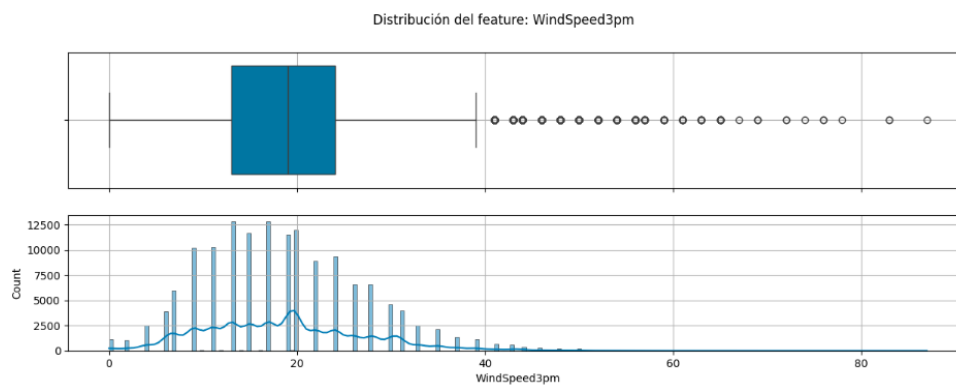


Figura 12

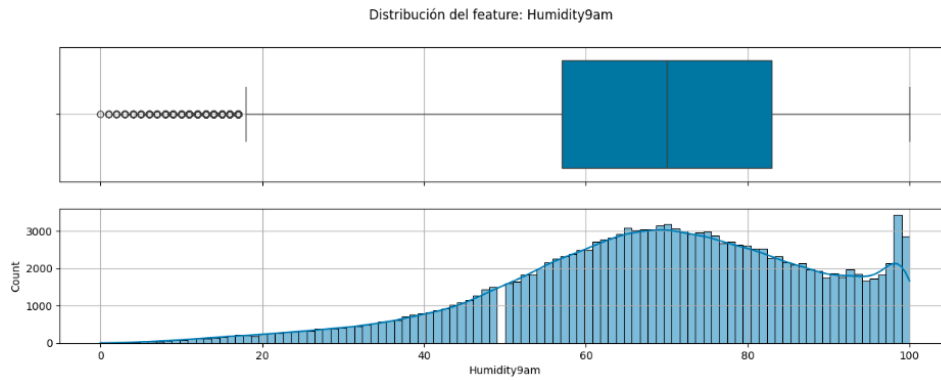


Figura 13

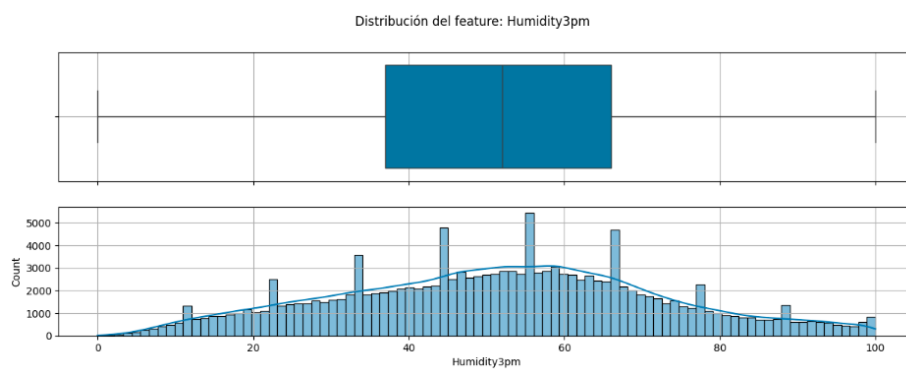


Figura 14

Correlación sin encodear (variables numéricas)

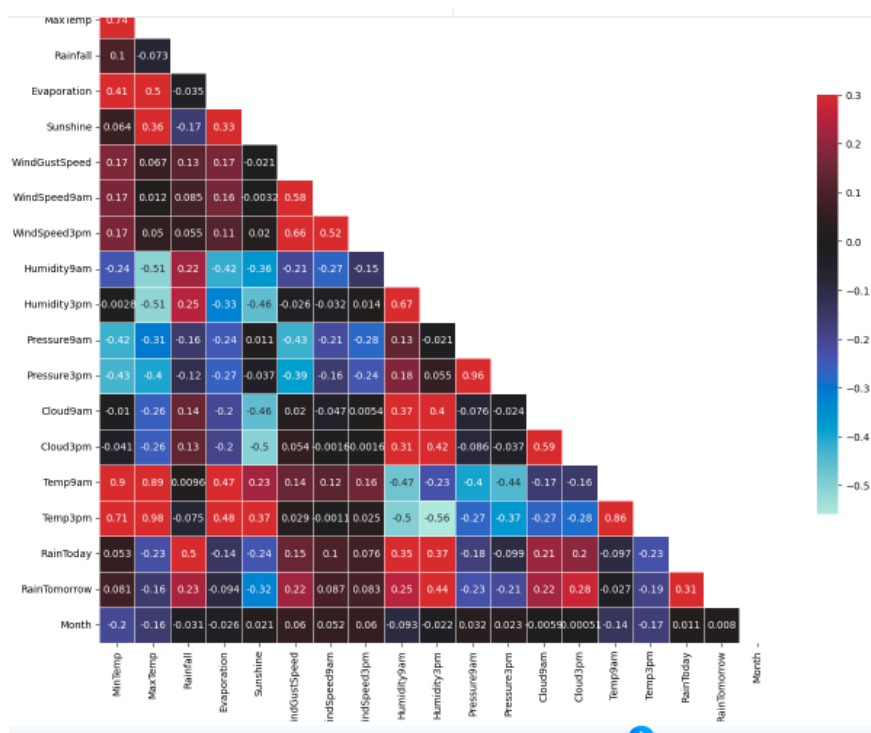


Figura 15: Correlación de Pearson utilizando solo variables numéricas.

Puede observarse que -en módulo- las correlaciones de mayor valor se dan entre la variable target y Rainfall, WindGustSpeed, Humidity9am, Humidity3pm, Cloud9am, Cloud3pm y RainToday.

Feature Engineering

En una primera iteración, se dispuso a encodear las variables categóricas ordinales en enteros representativos y las nominales se binarizaron en 0 y 1, quedando el dataset de la siguiente manera:

...	Location	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	WindDir3pm	Humidity9am	Humidity3pm	...	Cloud9am	Cloud3pm	1
0	62	0.6	4.6	8.6	270.0	44.0	270.0	292.5	71.0	22.0	...	8.0	5.0	
1	62	0.0	4.6	8.6	292.5	44.0	337.5	247.5	44.0	25.0	...	7.0	5.0	
2	62	0.0	4.6	8.6	247.5	46.0	270.0	247.5	38.0	30.0	...	7.0	2.0	
3	62	0.0	4.6	8.6	45.0	24.0	135.0	90.0	45.0	16.0	...	7.0	5.0	
4	62	1.0	4.6	8.6	270.0	41.0	67.5	315.0	82.0	33.0	...	7.0	8.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
145455	23	0.0	4.6	8.6	90.0	31.0	135.0	67.5	51.0	24.0	...	6.0	3.5	
145456	23	0.0	4.6	8.6	337.5	22.0	135.0	0.0	56.0	21.0	...	6.0	3.5	
145457	23	0.0	4.6	8.6	0.0	37.0	135.0	292.5	53.0	24.0	...	6.0	3.5	
145458	23	0.0	4.6	8.6	135.0	28.0	157.5	0.0	51.0	24.0	...	3.0	2.0	
145459	23	0.0	4.6	8.6	90.0	34.0	112.5	112.5	62.0	36.0	...	8.0	8.0	

145460 rows x 21 columns

Figura 16: Dataset a utilizar en una primera iteración con variables categóricas encodeadas.

El dataset final a utilizar en la primera iteración cuenta con 113.516 filas correspondientes a días sin lluvias en el día de mañana y 31.944 filas correspondientes a días que si hubo lluvia en día posterior. Se dispuso a balancear las clases en el dataset, de modo tal de contar con las 31.944 filas de días de lluvias al día posterior, y otras 31.944 filas elegidas al azar entre las 113.516 de días sin lluvias. El dataset a utilizar para entrenar los modelos posee finalmente 63.888 filas y 21 columnas:

	Location	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	WindDir3pm	Humidity9am	Humidity3pm	...	Cloud9am	Cloud3pm
37921	60	0.0	8.8	13.2	67.5	39.0	67.5	67.5	50.0	21.0	...	5.0	2.0
37871	60	0.0	5.0	11.7	292.5	54.0	67.5	315.0	49.0	15.0	...	0.0	1.0
52220	82	0.0	4.6	8.6	135.0	24.0	157.5	247.5	26.0	54.0	...	6.0	5.0
40081	78	0.0	1.8	7.8	292.5	31.0	292.5	315.0	73.0	44.0	...	1.0	3.0
43143	71	0.0	4.6	8.6	180.0	43.0	202.5	157.5	59.0	57.0	...	8.0	6.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
94184	52	0.0	3.6	10.3	45.0	35.0	157.5	67.5	52.0	54.0	...	0.0	1.0
66098	73	0.0	2.2	5.0	0.0	61.0	0.0	0.0	83.0	69.0	...	4.0	7.0
4171	65	0.2	4.6	8.6	202.5	37.0	225.0	225.0	69.0	82.0	...	6.0	5.0
74227	48	0.4	4.6	8.6	270.0	46.0	22.5	292.5	88.0	90.0	...	6.0	5.0
99342	69	0.0	8.7	12.6	22.5	30.0	45.0	67.5	34.0	18.0	...	6.0	5.0

63888 rows × 21 columns

Figura 17: Dataset balanceado listo para utilizar en el modelado.

Correlación con variables encodeadas

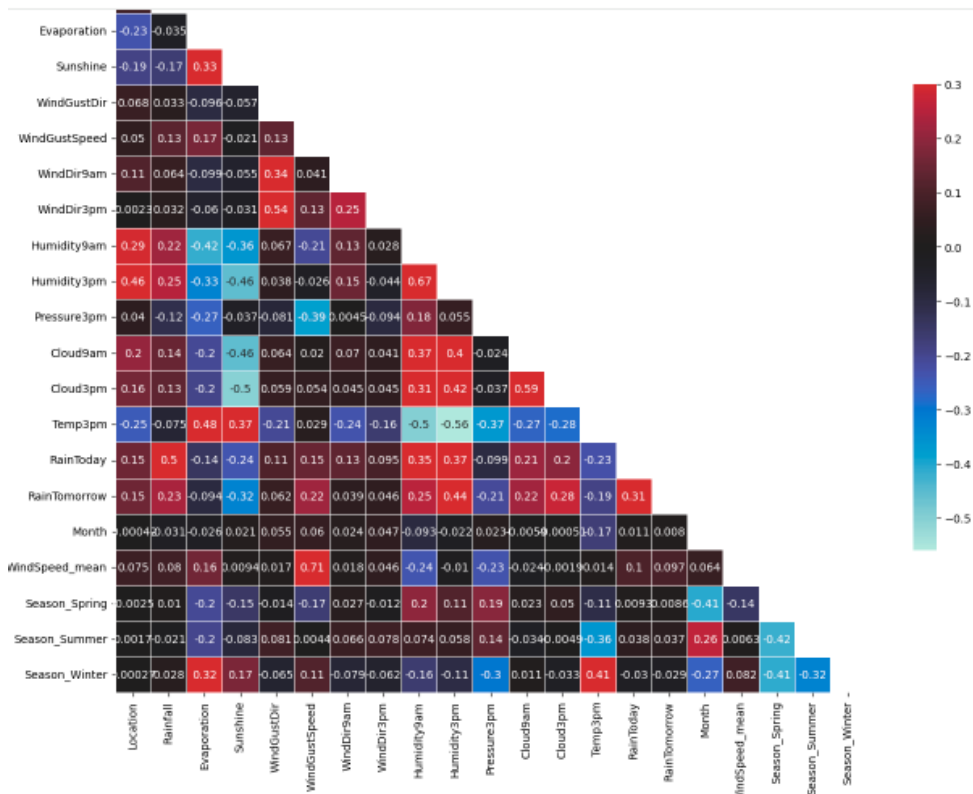


Figura 18: Correlación de Pearson utilizando todas las variables.

Baseline

Como primera iteración se entrenarán dos modelos: un Árbol de Decisión clásico y un Random Forest, ambos sin optimizar hiperparámetros ni aplicar validación cruzada. Se generarán tres (3) conjuntos de datos: uno para entrenamiento (train), otro para validación (val) y otro para testeo (test) del mejor modelo. El conjunto de prueba se reservará y no será utilizado hasta haber seleccionado el modelo más adecuado mediante el conjunto de validación.

Test - Train - Validation Split

Se destinará un 80% del dataset para entrenamiento y un 20% para el testeo. El 10% del set de entrenamiento será destinado para el set de validación.

Métricas

El modelo que mejor performó fue el Random Forest con su configuración estándar, obteniéndose un accuracy de 0.78 y un recall de 0.74 para la clase 1.

```
% de aciertos sobre el set de test: 0.7834559398966975
```

	precision	recall	f1-score	support
0	0.76	0.82	0.79	6332
1	0.81	0.74	0.78	6446
accuracy			0.78	12778
macro avg	0.79	0.78	0.78	12778
weighted avg	0.79	0.78	0.78	12778

Figura 19: Classification report del modelo.

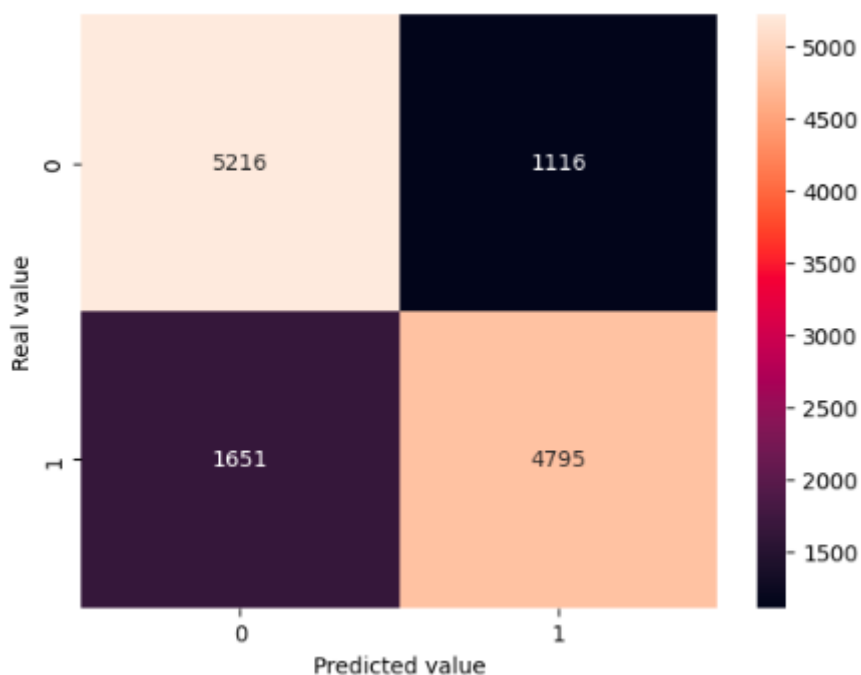


Figura 20: Matriz de confusión obtenida.

Puede observarse que los falsos positivos (FP), es decir, los valores que fueron predichos como positivos pero que en realidad no lo son, son menores a los falsos negativos (FN) que son los valores que fueron predichos como negativos, pero en realidad son positivos. Si bien esto no condice exactamente con lo buscado para la clase 1, será mejorada en las próximas iteraciones.

Conclusiones primera iteración

El presente trabajo fue realizado bajo el concepto de estudio de factibilidad, realizando una única iteración sobre las variables elegidas, los motivos y la escueta selección de modelos. Aun así, se concluye que la clasificación es factible de ser realizada, habiendo alcanzado valores de métricas altos, susceptibles de ser utilizados.

Segunda iteración

Habiendo obtenido un baseline en la primera iteración que, valida la hipótesis de factibilidad de la clasificación, se realizará una segunda iteración utilizando otros modelos y mejorando los hiperparámetros empíricamente de los ya utilizados buscando evitar el overfitting y obtener el mejor trade off sesgo-varianza posible para los más performantes. Se incluirán más métricas comparativas como la entropía cruzada binaria como función de pérdida, el área bajo la curva, y el gráfico de la curva ROC punto por punto, buscando optimizar la matriz de confusión en función de los requerimientos deseados, viendo si realmente el punto de corte por default para la clasificación binaria (≥ 0.5 utilizando la función sigmoide) es realmente el punto de corte ideal que maximiza el TPR (recall), que - junto con el accuracy, son las dos principales métricas de negocio a optimizar.

Las nuevas métricas y modelos se evaluarán bajo la misma estrategia de encodeo y feature engineering utilizada en la primera iteración, trabajando sobre el dataset balanceado obtenido en la primera entrega. Esta estrategia puede cambiar en la próxima y última iteración, dependiendo de los resultados obtenidos.

Modelos por implementar

Se probarán y optimizarán hiperparámetros sobre los siguientes modelos:

- **Árbol de decisión.**
- **Random forest.**
- **KNN.**
- **Regresión logística.**

Resultados obtenidos y comparativa de métricas

Función de pérdida sobre Train y Val

	Train	Val
Árbol de decisión	0.48	0,50
Random Forest	0.12	0.44
KNN	0.43	0.73
Regresión logística	0.48	0.47

Tabla 1: entropía cruzada obtenida sobre todos los sets de entrenamiento y validación.

Gracias a la entropía cruzada, es posible inferir -controlando previamente los hiperparámetros para evitar el *overfitting*- que el modelo que mejor capacidad de inferencia posee es el Random Forest con 45 estimadores.

Accuracy sobre Train, Val y Test

	Train	Val	Test
Árbol de decisión	0,77	0,76	0,76
Random Forest	0,99	0,80	0,80
KNN	0,79	0,75	0,75

	Train	Val	Test
Regresión logística	0,77	0,78	0,77

Tabla 2: Exactitud obtenida sobre todos los sets.

En esta tabla es posible observar los beneficios de haber optimizado los hiperparámetros sobre el set de validación. Se observa un incremento en la exactitud de todos los modelos utilizados respecto al baseline sobre el set de test, siendo el Random Forest nuevamente el que mejor performa.

Recall sobre Val y Test (clase 1)

	Val	Test
Árbol de decisión	0,75	0,75
Random Forest	0,80	0,80
KNN	0,72	0,72
Regresión logística	0,76	0,75

Tabla 3: TPR (Recall) obtenido sobre los sets de validación y testeo.

Se observa que nuevamente el Random Forest es el modelo que maximiza el Recall, siendo el modelo que mejor minimiza los falsos negativos.

ROC/AUC sobre Test incluyendo umbrales

En el notebook adjunto se incluye además las gráficas y los umbrales sobre el set de validación, así como también otras métricas como el F1-score que no se incluyeron en el presente informe por priorizar otras.

Árbol de decisión

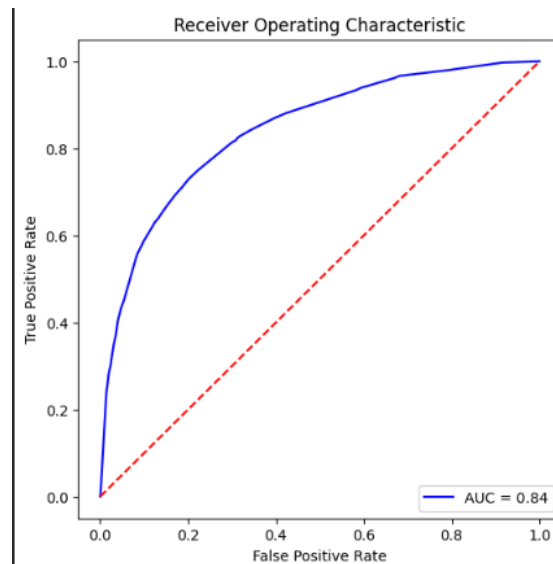


Figura 21: ROC/AUC sobre test utilizando el Árbol de decisión.

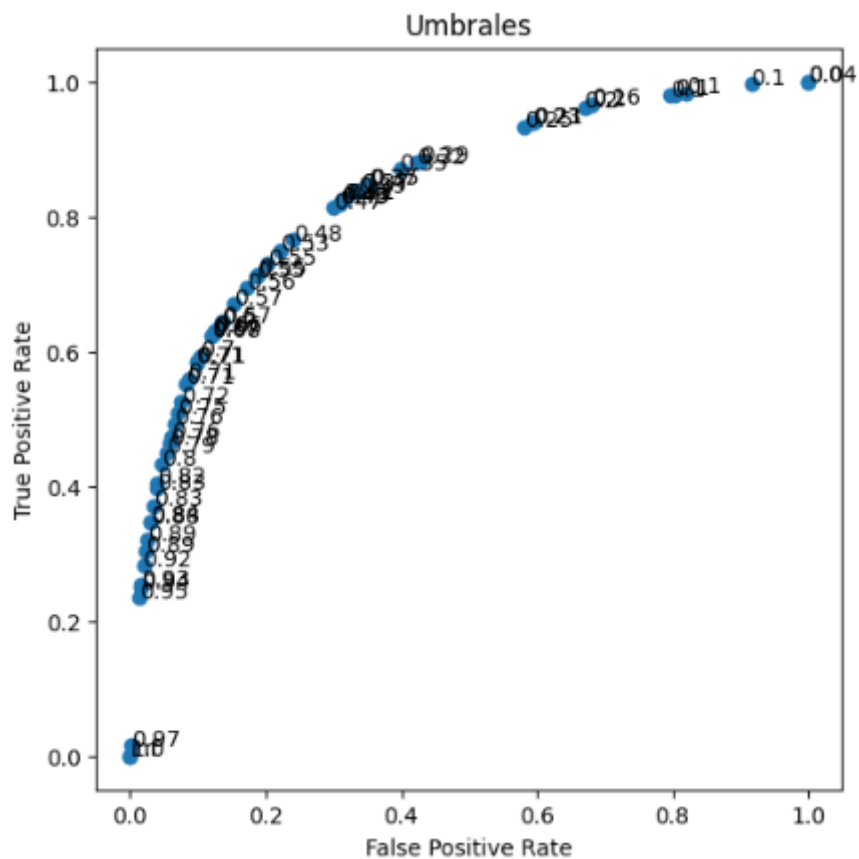


Figura 22: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando el Árbol de decisión.

Random Forest

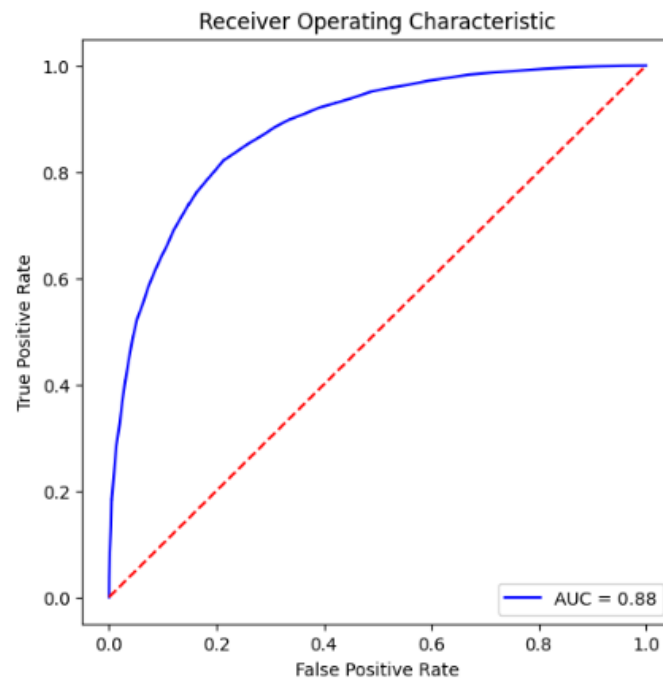


Figura 23: ROC/AUC sobre test utilizando Random Forest.

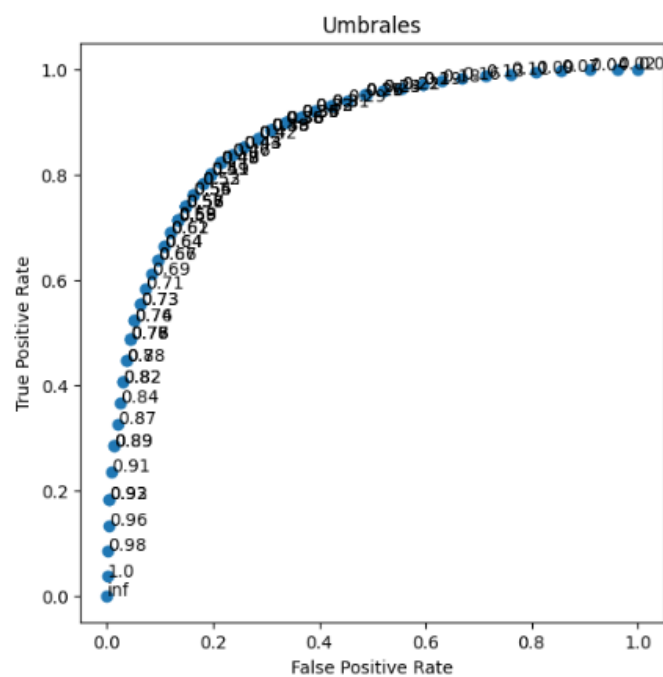


Figura 24: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando Random Forest.

KNN

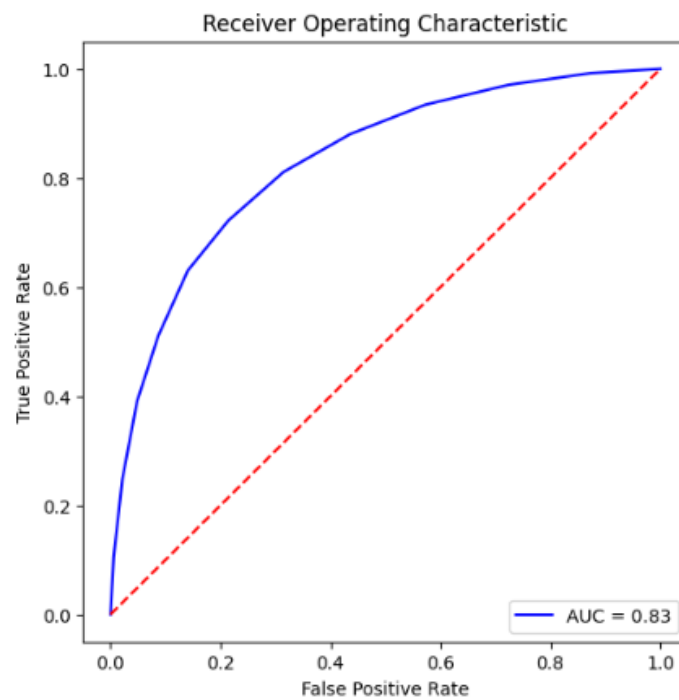


Figura 25: ROC/AUC sobre test utilizando KNN.

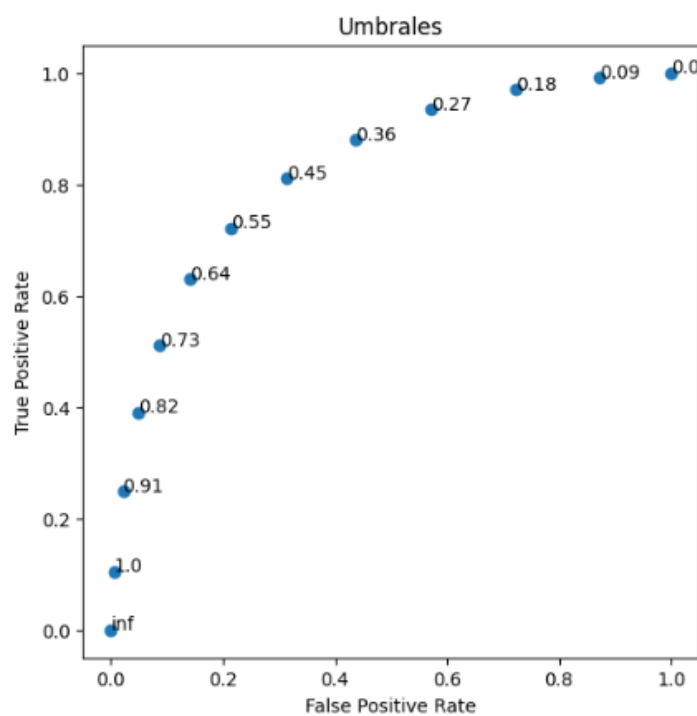


Figura 26: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando KNN.

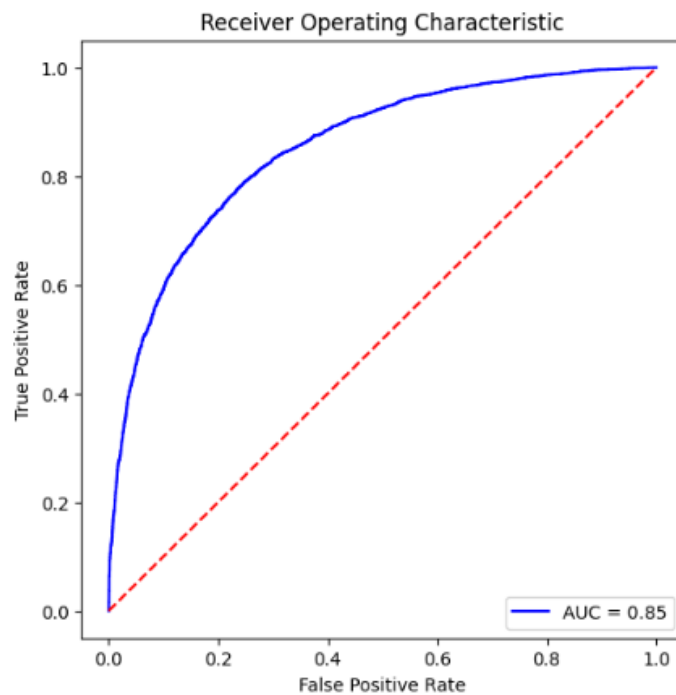


Figura 27: ROC/AUC sobre test utilizando Regresión logística.

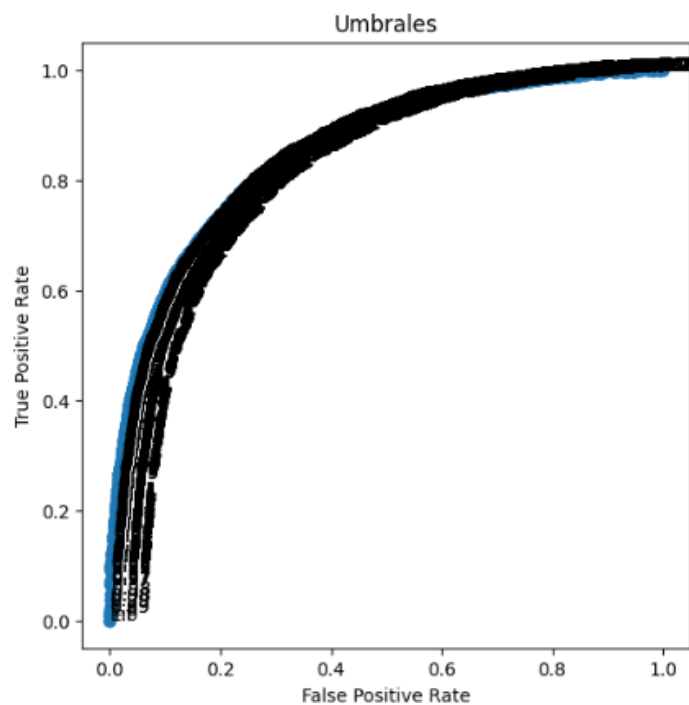


Figura 28: conjunto de umbrales correspondientes a todas las matrices de confusión de la ROC/AUC sobre test utilizando Regresión logística.

Se observa -nuevamente- que la mejor performance a la hora de clasificar y distinguir la variable target viene dada por el Random Forest. Será propósito de otro apartado el obtener aquella matriz de confusión que maximice el TPR, siendo una de nuestras principales métricas de negocio, sin descuidar el accuracy.

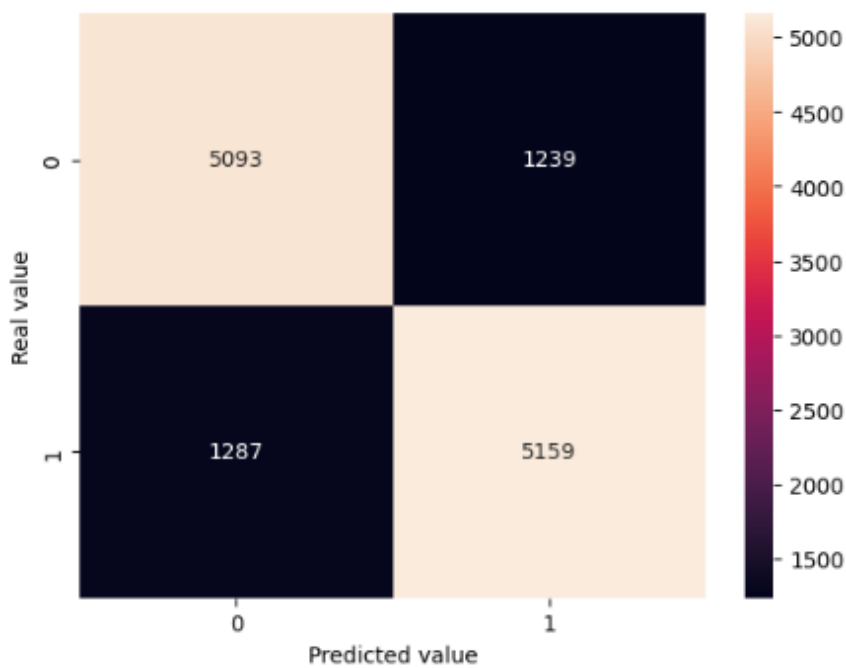


Figura 29: Matriz de confusión obtenida utilizando el modelo Random Forest mediante la optimización de estimadores.

Análisis PCA

Buscando optimizar los resultados obtenidos hasta el momento, se realizó una reducción de dimensionalidad, buscando como objetivo obtener el 95% de varianza en la menor cantidad de features o variables independientes posible, dando como resultado diez de las once variables presentes. Se probó este approach en los 2 modelos más performantes sobre el set de test: Random Forest y Regresión logística. Los resultados obtenidos fueron los siguientes:

	Accuracy	Recall	AUC
Random Forest	0.75	0,79	0,86
Regresión logística	0,77	0,75	0,85

Tabla 4: Métricas obtenidas mediante el análisis PCA.

Puede visualizarse que la performance de ambos modelos bajo esta premisa es menor, por lo cual se seguirá trabajando bajo la hipótesis principal, buscando mejorar las métricas existentes.

Corrección del umbral (optimal cutoff)

En la siguiente etapa se buscará -basándonos en el modelo que mejor performa- incrementar las métricas de negocio realizando una corrección al umbral definido por default intrínsecamente en el modelo al predecir. Se evaluarán aquellas matrices de confusión que maximicen el TPR (Recall) sin poner en desmedro el accuracy obtenido hasta el momento, pero teniendo en cuenta que un pequeño aumento en los falsos positivos no nos generaría una baja en la performance del modelo per se. Se analizará la curva ROC de Random Forest desde la derecha hacia la izquierda sobre el set de validación, buscando el umbral óptimo.

Trazando una analogía entre el TPR y el beneficio, como así también entre el FPR¹ y el costo, es posible generar una función susceptible de ser maximizada.

TPR - FPR $\Rightarrow$ máx.

	tpr	fpr	thr	tpr-fpr
30	0.800561	0.199311	0.511111	0.601249
29	0.779736	0.180566	0.515555	0.599170
28	0.779335	0.180566	0.533333	0.598769
26	0.756908	0.161438	0.555556	0.595470
27	0.756908	0.161821	0.549628	0.595087

Figura 30: valor máximo obtenido de la función a maximizar.

Corrigiendo la clase predicha por este nuevo umbral de probabilidad, se obtienen métricas idénticas a las anteriores, tanto en el set de validación como en el set de test, indicando que es lo mismo para este set de valores, estimar la cota inferior en 0.5 que en 0.525.

¹ FPR = FP / (FP + TN)

Mejoras para iteraciones posteriores

A continuación, se enuncian tres ideas para optimizar los resultados obtenidos:

1. Optimizar los hiperparámetros de los modelos que mejor performan de forma automática, mediante Gridsearch, abarcando más parámetros que los vistos en la presente entrega.
2. Implementar XGBoost con hiperparámetros optimizados.
3. Optimizar el flujo de trabajo del nosocomio ideando una matriz de confusión de 2 filas por 3 columnas, incorporando una semaforización ternaria mediante la parametrización de los umbrales, donde se puede estar enfermo, se puede ser un caso dudoso, o se puede estar sano, configurando un porcentaje destinado a la población dudosa.

Conclusiones segunda iteración

Se mejoraron todas las métricas obtenidas respecto al baseline presentado en la primera entrega aplicando la optimización manual de hiperparámetros logrando un **accuracy de 0.80 y un recall de 0.80** para la clase 1. Se comprobó a su vez que tanto una reducción en la dimensionalidad como un cambio en el umbral de cutoff no varían la performance del Random Forest, siendo éste el mejor modelo obtenido hasta el momento.

Modelo óptimo	Accuracy	Recall	AUC	F1
Random Forest	0,80	0,80	0,88	0,80

Tabla 5: Métricas óptimas obtenidas.

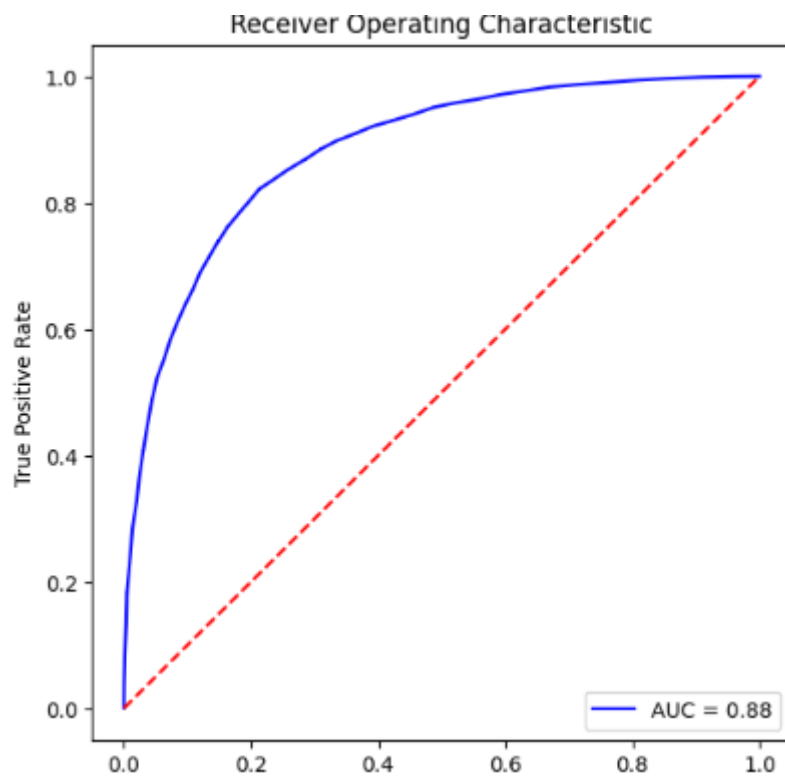


Figura 31: curva ROC obtenida para Random Forest.

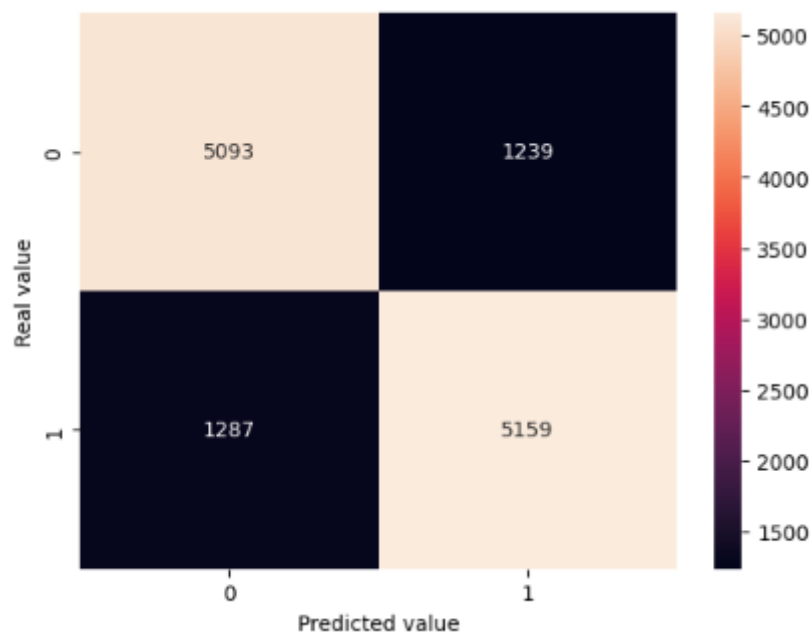


Figura 32: matriz de confusión obtenida.

Tercera iteración

En esta tercera iteración se buscará mejorar la performance de los modelos utilizados en la anterior a través de la optimización de hiperparámetros a través de un del uso de validación cruzada mediante **GridSearchCV** y **RandomizedSearchCV**. Se utilizará **StratifiedKFold** CV para evaluar los modelos optimizados, buscando que en cada iteración o fold haya la misma cantidad de elementos pertenecientes a ambas clases.

Se implementarán también modelos ensamble y boosting buscando mejorar las métricas logradas en iteraciones anteriores.

Modelos optimizados y métricas

GridSearchCV utilizando StratifiedKFold

Se fusionaron los sets de train y validación para la optimización de hiperparámetros ya que se va validando -valga la redundancia- con cada fold. Se obtuvieron los siguientes resultados sobre los modelos anteriormente analizados, utilizando 5 (cinco) folds y optimizando accuracy:

Árbol de decisión

```
Fitting 5 folds for each of 54 candidates, totalling 270 fits  
{'ccp_alpha': np.float64(0.0), 'criterion': 'gini', 'max_depth': 10, 'min_samples_leaf': np.int64(10), 'min_samples_split': np.int64(2)}
```

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0.763	0,004	0,805	0,769	0,747

Tabla 6: Métricas obtenidas mediante la optimización de parámetros.

Puede observarse en el notebook adjunto que las métricas a lo largo de los cinco splits se mantuvieron consistentes, generando un modelo robusto con un pequeño desvío estándar. Las métricas alcanzadas no son suficientes para reemplazar el mejor modelo obtenido, ya que su performance es menor al mismo sobre datos desconocidos.

Random Forest

```
Fitting 5 folds for each of 16 candidates, totalling 80 fits
{'bootstrap': True, 'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 10, 'n_estimators': np.int64(100)}
```

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0,795	0,004	0,958	0,803	0,803

Tabla 7: Métricas obtenidas mediante la optimización de parámetros.

El modelo es consistente con las métricas y -bajo esta configuración de hiperparámetros- es el que mejor performó sobre datos desconocidos teniendo en cuenta dos métricas importantes de negocio (Accuracy y Recall). No obstante, estos valores se encuentran alejados de los obtenidos en la iteración anterior.

KNN

```
Fitting 5 folds for each of 6 candidates, totalling 30 fits
{'metric': 'manhattan', 'n_neighbors': 15, 'weights': 'distance'}
```

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0.781	0,002	0,999	0,785	0,764

Tabla 8: Métricas obtenidas mediante la optimización de parámetros.

Si bien el modelo es constante con las métricas en cada fold, obteniendo un bajo desvío estándar, obtiene bajos resultados a la hora de generalizar sobre datos desconocidos.

```
Fitting 5 folds for each of 30 candidates, totalling 150 fits
{'C': np.float64(0.019306977288832496), 'penalty': 'l2', 'solver': 'liblinear'}
```

Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
0,778	0,005	0,779	0,779	0,767

Tabla 9: Métricas obtenidas mediante la optimización de parámetros.

El modelo performa correctamente sobre el set de test, y continúa siendo el segundo mejor modelo a elegir para resolver esta clasificación. Las métricas obtenidas mediante esta técnica de optimización de hiperparámetros arrojó resultados levemente superiores a la iteración anterior, pero sin superar al Random Forest.

RandomizedSearchCV utilizando StratifiedKFold

Modelo	Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
Árbol de decisión	0,763	0,004	0,805	0,769	0,747
Random Forest	0,795	0,005	0,933	0,802	0,798
KNN	0,781	0,002	1,0	0,785	0,764
Regresión Logística	0,778	0,004	0,778	0,781	0,768

Tabla 10: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV.

Tras realizar la optimización de hiperparámetros mediante RandomizedSearchCV, Random Forest demostró ser el modelo de mejor performance global. Su arquitectura de ensamble le permite mantener

métricas equilibradas y realistas, evitando el sobreajuste y garantizando una predicción más fiable ante nuevos datos de lluvia. Los hiperparámetros finales resultantes de la búsqueda se detallan en el notebook correspondiente.

Modelos boosting

Buscando mejorar las métricas obtenidas hasta el momento, se implementarán 4 (cuatro) modelos de boosting utilizando sus parámetros por default. Los valores obtenidos fueron los siguientes:²

Modelo	Acc train	Acc val	Acc test	Recall val (clase 1)	Recall test (clase 1)	Entropía cruzada train	Entropía cruzada val	AUC test
AdaBoost	0,77	0,77	0,77	0,77	0,78	0,60	0,57	0,86
Gradient Boosting	0,79	0,79	0,79	0,78	0,78	0,44	0,44	0,87
XGBoost	0,87	0,79	0,80	0,79	0,80	0,31	0,43	0,88
LightGBM	0,82	0,80	0,80	0,80	0,80	0,31	0,43	0,88

Tabla 11: Comparativa de métricas entre modelos boosting implementados.

De los modelos analizados, resaltan por su performance sobre todos los indicadores **XGBoost y LightGBM**. Si bien los modelos minimizan la entropía cruzada entre train y validación, obtiene métricas similares al Random Forest obtenido en la iteración anterior.

² Las curvas ROC, los umbrales y las matrices de confusión se adjuntan en el notebook para no sobrecargar la presentación.

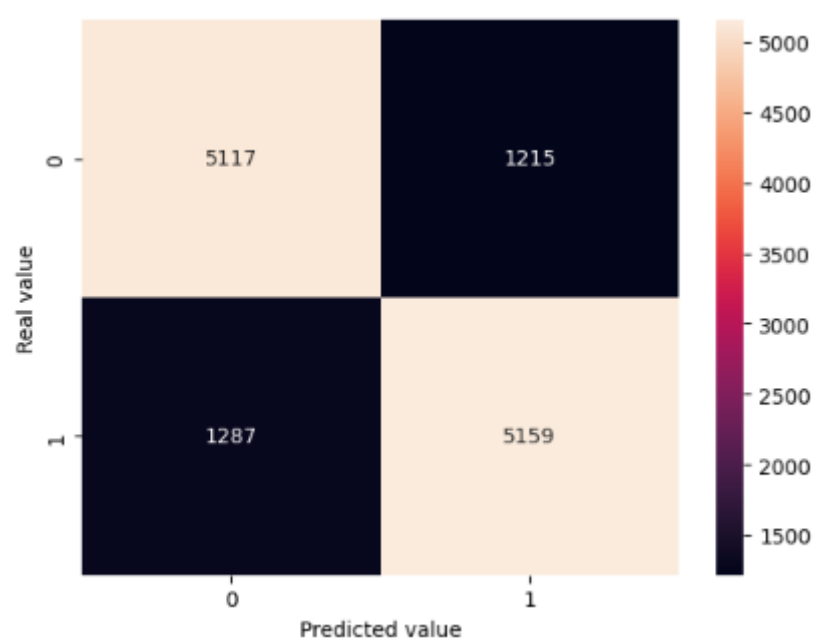


Figura 33: matriz de confusión obtenida mediante XGBoost sobre el set de test.

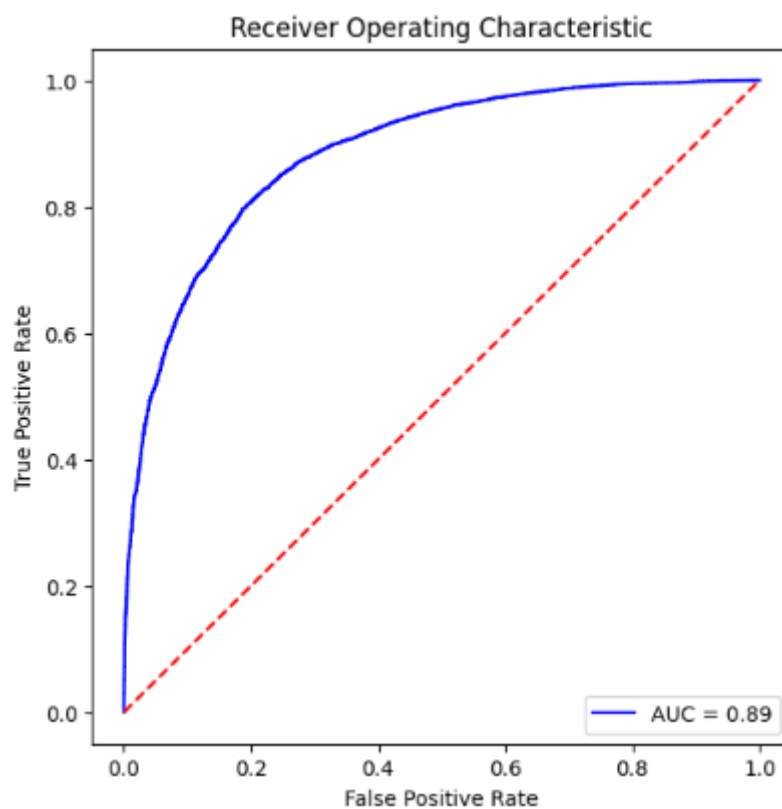


Figura 34: curva ROC obtenida para XGBoost sobre el set de test.

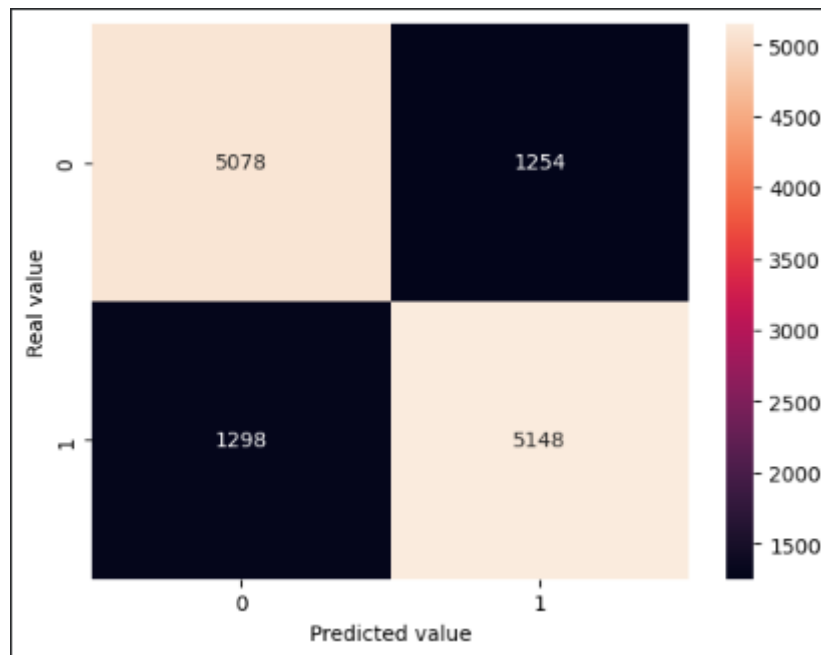


Figura 35: matriz de confusión obtenida mediante LightGBM sobre el set de test.

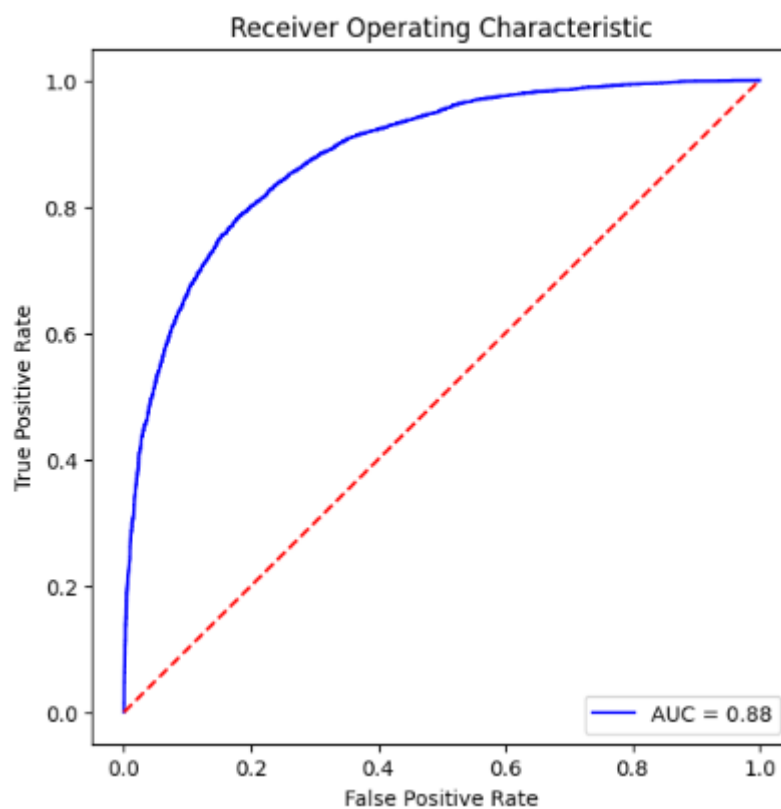


Figura 36: curva ROC obtenida para LightGBM sobre el set de test.

Puede observarse en las figuras 33 y 35 que tanto el Accuracy, el Recall y Precision son menores que el modelo obtenido anteriormente.

Modelos boosting optimizados

Se optimizaron los 4 (cuatro) modelos utilizados anteriormente mediante RandomizedSearchCV utilizando StratifiedKFold. Se buscará generar una grilla de parámetros lo más variada posible. Los resultados obtenidos fueron los siguientes:

Modelo	Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
AdaBoost	0,779	0,003	0,784	0,781	0,760
Gradient Boosting	0,795	0,004	0,806	0,797	0,789
XGBoost	0,823	0,002	0,962	0,805	0,806
LightGBM	0,771	0,003	0,774	0,775	0,764

Tabla 12: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV sobre los modelos boosting.

De los modelos optimizados, resalta por su performance sobre todos los indicadores el **XGBoost**. Si bien el modelo obtenido performa muy bien, no es capaz de superar al Random Forest obtenido en la iteración anterior.

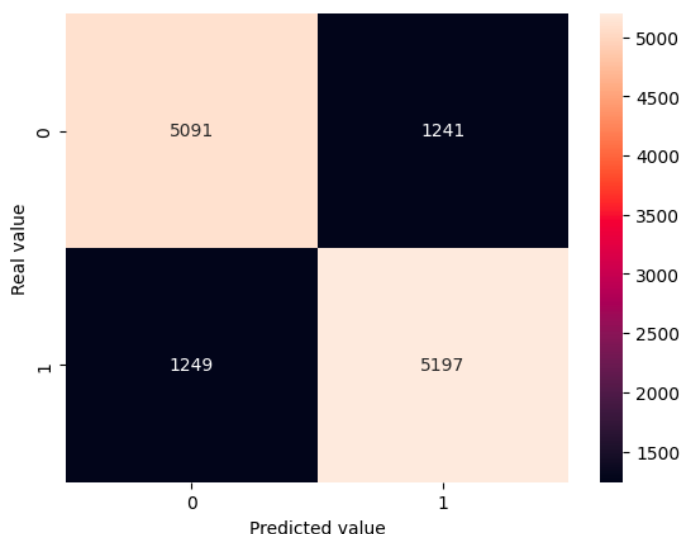


Figura 37: matriz de confusión obtenida mediante XGBoost optimizado sobre el set de test.

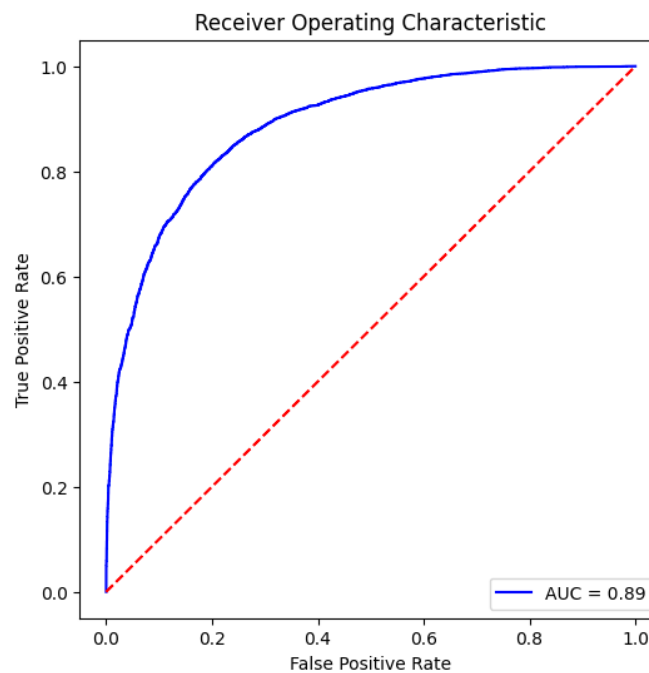


Figura 38: curva ROC obtenida para XGBoost optimizado sobre el set de test.

Puede observarse en la figura 37 que tanto el Accuracy como el Recall son menores que el modelo obtenido en la iteración anterior.

Bagging (Bootstrap aggregating)

Se implementará un **BaggingClassifier** con árboles de decisión debido a que son los modelos que más varianza o inestabilidad presentan. Se buscará el número de estimadores ideal empíricamente, evitando que el modelo overfitee. Los resultados obtenidos fueron los siguientes:

Modelo	Acc train	Acc val	Acc test	Recall val (clase 1)	Recall test (clase 1)	Entropía cruzada train	Entropía cruzada val	AUC test
Bagging Classifier	0,99	0,78	0,78	0,75	0,76	0,12	0,79	0,86

Tabla 13: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV sobre los modelos boosting.

Nuevamente, el modelo obtenido no logra superar las mejores métricas de negocio obtenidas hasta el momento.

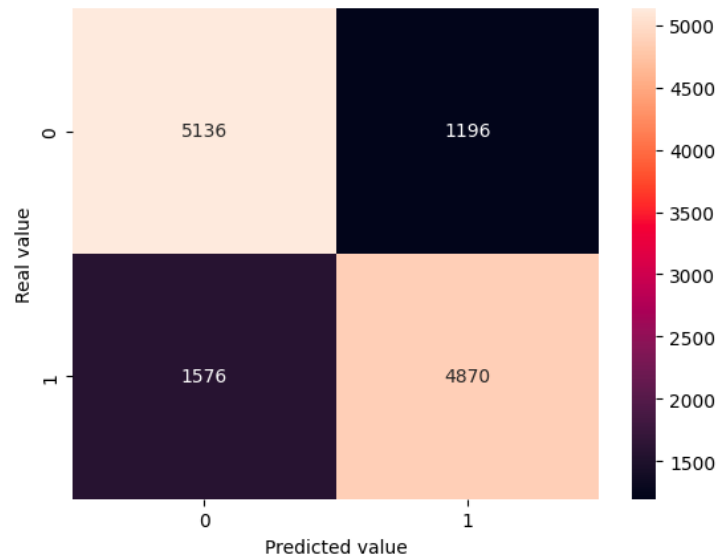


Figura 39: matriz de confusión obtenida mediante Bagging Classifier sobre el set de test.

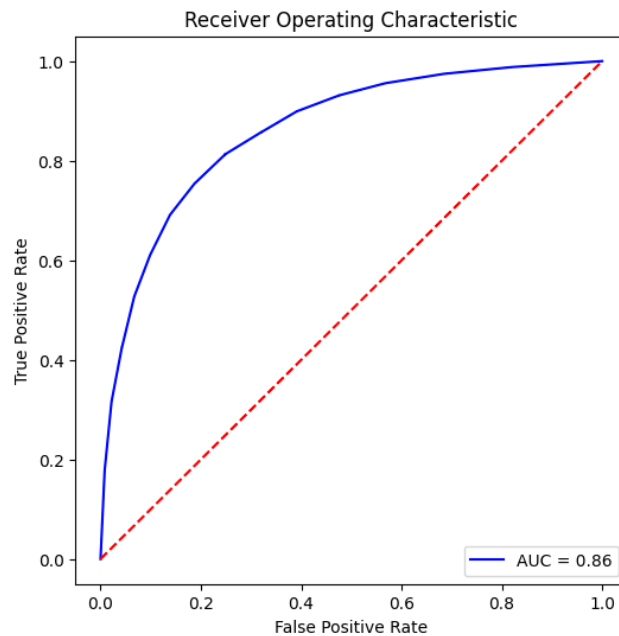


Figura 40: curva ROC obtenida para Bagging Classifier sobre el set de test.

Bagging optimizado

Se optimizarán los hiperparámetros tanto del estimador base utilizado (Árbol de decisión) como del Bagging Classifier propiamente dicho mediante el uso de **RandomizedSearchCV** en conjunto con **StratifiedKFold**.

Modelo	Accuracy promedio	Desvío estándar	Accuracy train	Accuracy test	Recall test
Bagging Classifier	0,788	0,006	0,831	0,794	0,786

Tabla 14: Métricas obtenidas mediante la optimización de parámetros utilizando RandomizedSearchCV sobre Bagging Classifier y su estimador base.

El modelo optimizado performa de manera muy similar al optimizado empíricamente, con la salvedad de que se ve penalizada levemente una métrica importante de negocio como lo es el Recall o TPR, lo cual puede visualizarse en la matriz de confusión adjuntada:

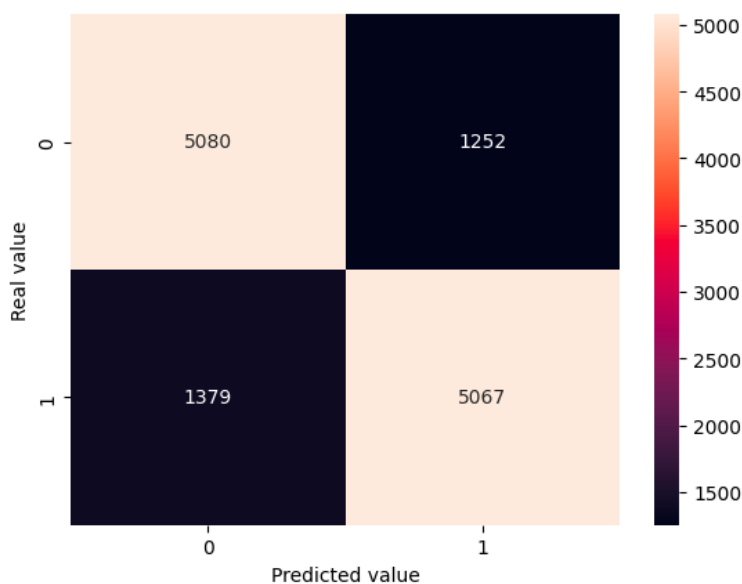


Figura 41: matriz de confusión obtenida mediante Bagging Classifier optimizado sobre el set de test.

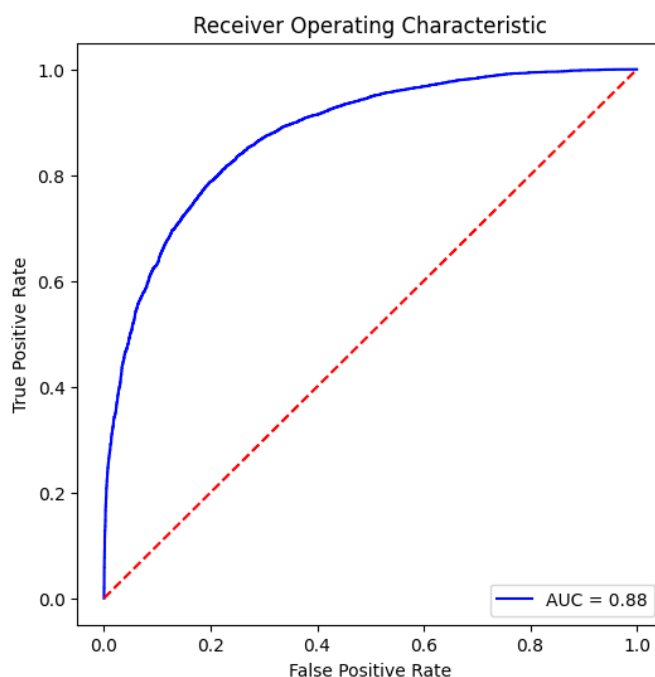


Figura 42: curva ROC obtenida para Bagging Classifier optimizado sobre el set de test.

Conclusiones tercera iteración

Tras evaluar los modelos de ensamble avanzados en la tercera iteración, como **XGBoost** y **LightGBM**, se observa que sus métricas de desempeño (**Accuracy 0.80** y **Recall 0.80**) resultan equivalentes a las obtenidas por el **Random Forest** de la segunda iteración.

Ante la paridad de resultados, se opta por el **Random Forest optimizado empíricamente** como el modelo óptimo. Esta decisión se fundamenta en que dicho modelo alcanzó la convergencia máxima de las métricas de negocio con una menor complejidad estructural, demostrando una mayor robustez y capacidad de generalización sobre datos desconocidos. Al haber llegado al límite de predictibilidad de los datos actuales, se prioriza la estabilidad del Random Forest frente al riesgo de sobreajuste (overfitting) que presentan los modelos de boosting más complejos.

Modelo óptimo	Accuracy	Recall	AUC	F1
Random Forest	0,80	0,80	0,88	0,80

Tabla 15: Métricas principales óptimas obtenidas sobre el set de test.

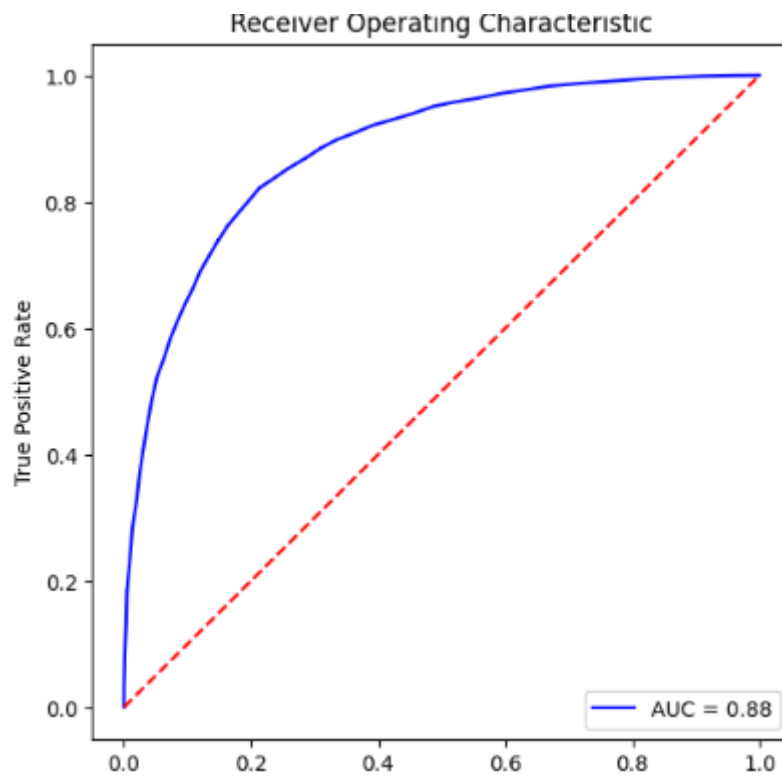


Figura 43: curva ROC óptima obtenida sobre el set de test.

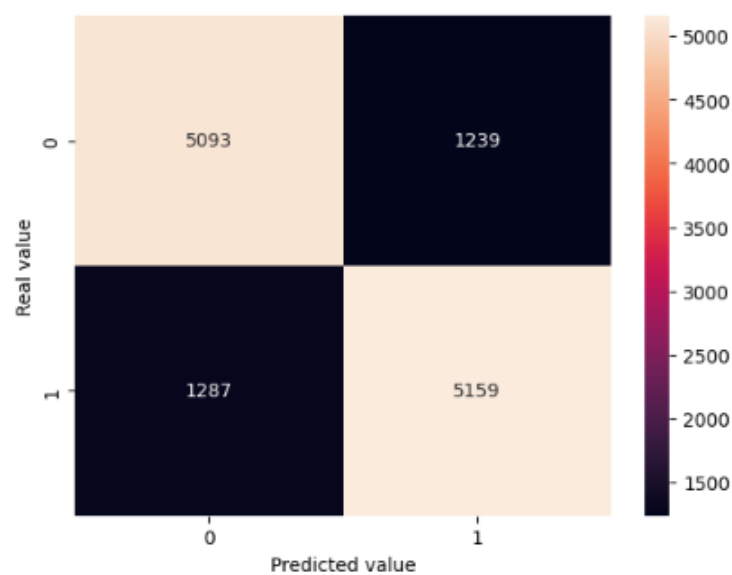


Figura 44: matriz de confusión óptima obtenida sobre el set de test.